

Направление: 09.03.01 «Информатика и вычислительная техника»

Профиль: «Математические основы искусственного интеллекта»

ОТЧЕТ ПО ПРОЕКТУ №2

по дисциплине

«Основы математического и компьютерного моделирования»

Тема проекта:

**«Расчет фрактальной размерности береговой
линии методом Box-Counting»**

Выполнили студенты:

Лакпажап Дан-Хаяа

Хадралинова Полина

Проверил:

д-р физ.-мат. наук, проф.

Масловская А.Г.

1 Прикладная задача и математическая постановка

1.1 Прикладная задача

Береговые линии являются классическим примером фрактальных объектов, длина которых зависит от масштаба измерения. Это свойство имеет прикладное значение в геоморфологии, экологии и береговой инженерии.

Целью работы является разработка численного метода оценки фрактальной размерности береговой линии по растровому изображению и анализ пространственной вариативности её сложности.

1.2 Классификация модели

- **Детерминированная:** результат однозначно определяется входным изображением и параметрами алгоритма.
- **Дискретная:** оперирует пиксельной матрицей и конечным набором масштабов сетки.
- **Статическая:** не описывает временную динамику объекта; береговая линия неизменна.
- **С использованием линейной аппроксимации:** оценка размерности основана на линейной регрессии в логарифмических координатах.
- **Сосредоточенная:** анализирует локальные участки независимо, не описывая пространственные физические процессы переноса.

1.3 Математическая модель

Фрактальная размерность по Минковскому (box-counting dimension) определяется как:

$$D = \lim_{\epsilon \rightarrow 0} \frac{\ln N(\epsilon)}{\ln(1/\epsilon)}, \quad (1)$$

где ϵ — размер стороны квадратной ячейки, а $N(\epsilon)$ — число ячеек, покрывающих множество.

1.4 Параметры модели

Параметр	Описание	Единицы измерения
D	Фрактальная размерность по Минковскому	Безразмерная
ϵ	Размер стороны ячейки сетки	Пиксели (px)
$N(\epsilon)$	Количество ячеек, покрывающих контур	Безразмерная
W	Размер скользящего окна	Пиксели (px)
I	Яркость пикселя изображения	Уровни яркости(бит)
x, y	Координаты точки на изображении	Пиксели (px)

Ограничения и требования: контраст яркости ≥ 40 единиц, ширина границы не должна превышать 2–3 пикселей, отсутствие текстурных артефактов (облаков, теней)

1.5 Выбор численного метода

Для оценки фрактальной размерности береговой линии по растровому изображению используется метод box-counting (Минковского). Выбор метода обусловлен его применимостью к бинарным изображениям, простотой численной реализации и устойчивостью к эффектам дискретизации.

Метод не требует явного выделения непрерывной кривой и допускает эффективную обработку больших изображений. Кроме того, он естественным образом расширяется на локальный анализ, что позволяет исследовать пространственное распределение фрактальной размерности.

В работе применяется модификация метода с фиксированным числом сдвигов сетки ($\text{max_shifts} = 2$), уменьшающая влияние квантования.

1.6 Формализация алгоритма

Алгоритм оценки фрактальной размерности включает следующие этапы:

Вход: бинарное изображение I размера $H \times W$.

Выход: глобальная фрактальная размерность D_{global} и карта локальных значений $D_{\text{local}}(x, y)$.

1. Построение набора масштабов $R = \{r_1, r_2, \dots, r_k\}$.
2. Для каждого масштаба $r \in R$:
 - разбиение изображения на сетку размера $r \times r$;
 - подсчёт числа занятых ячеек $N(r)$ с учётом сдвигов сетки;
3. Оценка D_{global} как наклона линейной регрессии зависимости $\ln N(r)$ от $\ln(1/r)$.
4. Для каждого пикселя контура:
 - извлечение локального окна;
 - повторение процедуры box-counting;
 - вычисление $D_{\text{local}}(x, y)$.

2 Программная реализация

Программный комплекс разработан на языке Python 3.12. Выбор данного стека обусловлен наличием высокопроизводительных библиотек для обработки изображений и матричных вычислений, что критично для алгоритмов с использованием скользящего окна.

2.1 Описание применения пакетных решений

В работе применены следующие ключевые возможности сторонних библиотек:

- **OpenCV**: используется функция `cv2.threshold` с флагом `THRESH_OTSU` для автоматического определения порога бинаризации и детектор `cv2.Canny` для выделения контура толщиной в 1 пиксель.
- **NumPy**: основной инструмент для матричных преобразований. Метод `reshape` применяется для разбиения изображения на сетку (боксы) без использования медленных вложенных циклов. Функция `np.polyfit` используется для реализации метода наименьших квадратов (МНК).

2.2 Исходный код программы с комментариями

```
1 import cv2
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 def box_counter(img_matrix, r):
6     """
7     Вучисляет количество непустых блоков N(r) для заданного масштаба r.
8     """
9     h, w = img_matrix.shape
10    pad_h, pad_w = (r - h % r) % r, (r - w % r) % r
11
12    if pad_h > 0 or pad_w > 0:
13        img_padded = np.pad(img_matrix, ((0, pad_h), (0, pad_w)), mode='
constant')
14    else:
15        img_padded = img_matrix
16
17    grid_h, grid_w = img_padded.shape[0] // r, img_padded.shape[1] // r
18    blocks = img_padded.reshape(grid_h, r, grid_w, r)
19
20    return np.count_nonzero(blocks.sum(axis=(1, 3)))
21
22 def get_min_n(img_matrix, r, max_shifts=2):
23     """
24     Определяет минимальное покрытие N(r) путем трансляции сетки (Continuous
25     Box-Counting).
26     """
27     min_n = float('inf')
28     shifts = min(r, max_shifts)
29
30     for dy in range(shifts):
31         for dx in range(shifts):
32             shifted = img_matrix[dy:, dx:]
33             if shifted.size == 0: continue
34
35             n = box_counter(shifted, r)
36             if n < min_n:
37                 min_n = n
38
39     return min_n
40
41 def calculate_fd(img_matrix, scales):
42     """
43     Аппроксимация фрактальной размерности Минковского методом наименьших ква
44     дратов.
45     """
46     n_values = []
47     valid_scales = []
48
49     for r in scales:
50         n = get_min_n(img_matrix, r)
51         if n > 0:
52             n_values.append(n)
53             valid_scales.append(r)
54
55     if len(n_values) < 2:
56         return 1.0
57
58     # Линейная регрессия
```

```

56     coeffs = np.polyfit(np.log(valid_scales), np.log(n_values), 1)
57     return -coeffs[0]
58
59 def process_coastline(image_path, win_sizes=[16, 32, 64], local_scales=np.
    array([2,4,8,16])):
60     """
61     Пространственный анализ фрактальных характеристик изображения.
62     """
63     img = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
64     if img is None:
65         return
66
67     # Предобработка и детекция границ
68     _, binary = cv2.threshold(img, 0, 255, cv2.THRESH_BINARY_INV + cv2.
    THRESH_OTSU)
69     edges = (cv2.Canny(binary, 100, 200) > 0).astype(np.uint8)
70
71     global_scales = np.unique(np.floor(np.logspace(0.5, 3, 10)).astype(int))
72     global_scales = global_scales[global_scales > 0]
73     global_fd = calculate_fd(edges, global_scales)
74
75     print(f"Dataset: {image_path} | Global D = {global_fd:.4f}")
76
77     # Размерности локально
78     for win_size in win_sizes:
79         h, w = edges.shape
80         heatmap = np.zeros((h, w), dtype=np.float32)
81         y_coords, x_coords = np.where(edges > 0)
82
83         pad = win_size // 2
84         padded_edges = np.pad(edges, pad, mode='constant')
85
86         for y, x in zip(y_coords, x_coords):
87             window = padded_edges[y : y + win_size, x : x + win_size]
88             heatmap[y, x] = calculate_fd(window, local_scales)
89
90     _visualize_results(img, edges, heatmap, win_size, global_fd,
    image_path)
91
92 def _visualize_results(img, edges, heatmap, win_size, global_fd, path):
93     vis_heatmap = cv2.dilate(heatmap, np.ones((3,3), np.uint8), iterations
    =2)
94
95     plt.figure(figsize=(10, 5))
96     plt.suptitle(f"Scale Analysis: {path} | Window: {win_size} | Global D: {
    global_fd:.3f}")
97
98     plt.subplot(1, 2, 1)
99     plt.imshow(img, cmap='gray')
100    plt.imshow(np.ma.masked_where(edges == 0, edges), cmap='spring', alpha
    =0.6)
101    plt.title("Detected Edge Map")
102    plt.axis('off')
103
104    plt.subplot(1, 2, 2)
105    masked_hm = np.ma.masked_where(vis_heatmap <= 0.5, vis_heatmap)
106    plt.imshow(img, cmap='gray', alpha=0.4)
107    plt.imshow(masked_hm, cmap='turbo', vmin=1.0, vmax=1.8)
108    plt.colorbar(label='Local Fractal Dimension (D)', shrink=0.7)
109    plt.title("FD Heatmap")

```

```

110     plt.axis('off')
111
112     plt.tight_layout()
113     plt.show()
114
115 for fname in ["photoname.png"]:
116     process_coastline(fname)

```

3 Анализ результатов

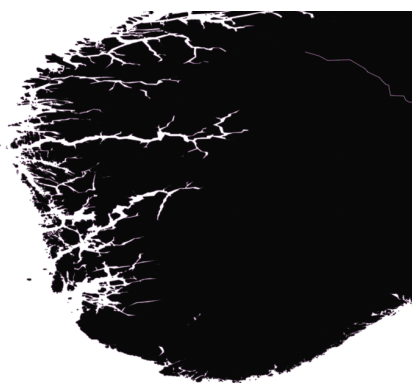
3.1 Верификация на эталонных объектах

Объект	Теория (D)	Эксперимент (D)	Погрешность
Снежинка Коха	1.2619	1.316	4.3%
Великобритания	1.2500	1.2492	0.1%
Норвегия	1.5200	1.4136	7.0%

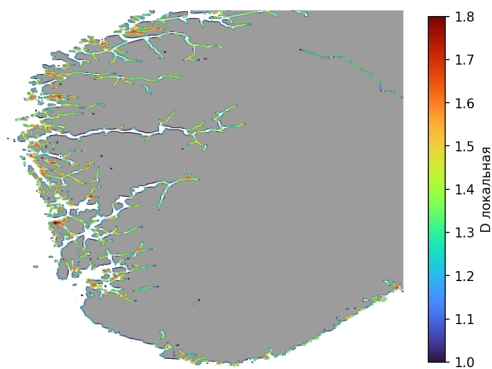
Примечание. Расхождение для Норвегии (7%) объясняется тем, что классическое значение $D \approx 1.52$ относится ко всему побережью страны. Наше изображение охватывает южный регион с умеренной изрезанностью. Таким образом, алгоритм корректно отражает региональные различия.

3.2 Влияние размера скользящего окна

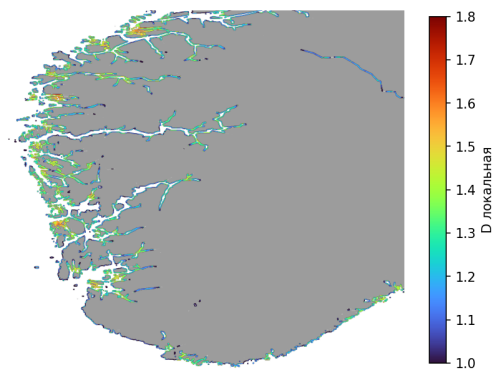
Для исследования устойчивости метода был проведён эксперимент с изменением размера скользящего окна $W \in \{16, 32, 64\}$.



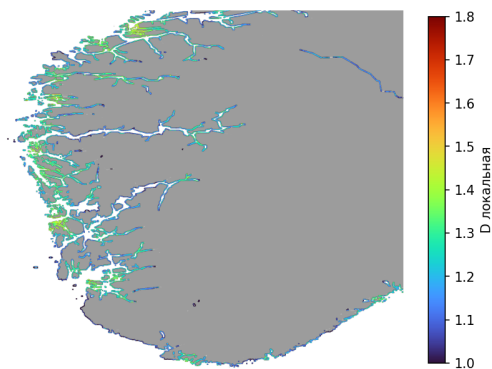
(a) Бинарный оригинал



(b) Окно $W = 16$



(c) Окно $W = 32$ (оптимум)

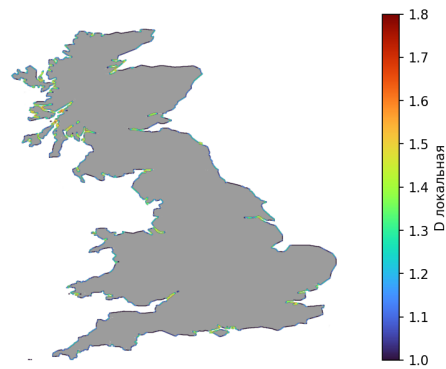


(d) Окно $W = 64$

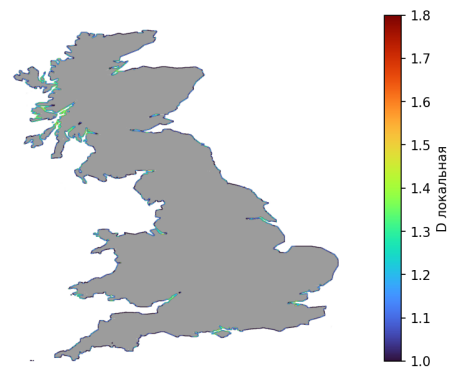
Рис. 1: Сравнение тепловых карт при различных размерах окна.



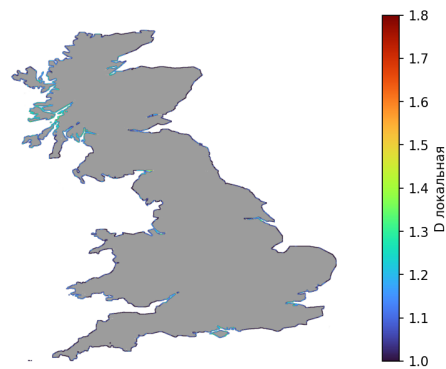
(a) Бинарный оригинал



(b) Окно $W = 16$



(c) Окно $W = 32$ (оптимум)



(d) Окно $W = 64$

Рис. 2: Сравнение тепловых карт при различных размерах окна.

3.3 Обсуждение и интерпретация результатов

1. **Адекватность модели:** Полученные значения для Великобритании ($D \approx 1.25$) и Норвегии ($D \approx 1.41$) согласуются с известными оценками и подтверждают применимость фрактальной размерности как показателя сложности береговой линии. Различие порядка 0.16 отражает различия в морфологии побережий.
2. **Пространственная неоднородность:** Тепловые карты выявляют неоднородное распределение локальной размерности. Для Великобритании повышенные значения ($D > 1.4$) локализованы на северо-западе, тогда как восточное побережье характеризуется меньшей сложностью ($D \approx 1.1$). Для Норвегии высокие значения распределены вдоль большей части побережья.
3. **Влияние масштаба (W):** При малых значениях окна ($W < 16$) возрастает влияние дискретизации, при больших ($W > 64$) наблюдается сглаживание локальных особенностей. Значение $W = 32$ обеспечивает компромисс между детализацией и устойчивостью оценок.
4. **Влияние сдвигов сетки:** Использование сдвигов сетки снижает погрешность оценки по сравнению с фиксированной сеткой, что указывает на необходимость учета эффектов квантования.

4 Заключение

В ходе работы разработан программно-алгоритмический метод оценки фрактальной размерности береговой линии на основе алгоритма box-counting с модификацией CMF.

Проведённые эксперименты на тестовых и реальных данных показали, что метод обеспечивает согласующиеся с литературными значениями оценки и позволяет различать типы береговых линий по уровню их изрезанности.

Использование скользящего окна позволяет выявлять пространственную неоднородность фрактальных характеристик береговой линии.

Возможные улучшения: дальнейшее развитие метода может включать оптимизацию вычислений за счёт параллельной обработки изображений, адаптивный выбор параметров сетки, а также интеграцию с векторными геоинформационными данными для повышения точности анализа.

Предложенный подход может быть применён в задачах геоморфологии, картографии и анализа спутниковых изображений.